

# Computer-Assisted Theorem Proving: 7CCSACTP

## Exercise Sheet 3

### Exercise 3.1 Fold function

The fold function is a very generic function, that can be used to express multiple other interesting functions over lists.

Have a look at Isabelle/HOL's standard function *fold*, defined as follows:

```
fun fold::('a  $\Rightarrow$  'b  $\Rightarrow$  'b)  $\Rightarrow$  'a list  $\Rightarrow$  'b  $\Rightarrow$  'b where  
fold f [] acc = acc  
| fold f (x #xs) acc = fold f xs (f x acc)
```

Write a function to compute the sum of the elements of a list. Define two versions, one direct recursive definition, and one using fold. Show that both are equal.

Hint: use automation!

```
fun list_sum :: nat list  $\Rightarrow$  nat
```

```
definition list_sum' :: nat list  $\Rightarrow$  nat
```

```
lemma list_sum xs = list_sum' xs
```

### Exercise 3.2 Tail recursive *reverse*

In Isabelle, there is the reverse function *rev*, which, given a list, computes another list with the same elements but in reverse order. Take a look into its definition:

```
thm rev.simps
```

Recall tail recursion: a function is tail recursive if in all recursive calls it does not call anything after itself<sup>1</sup>. For instance, the implementation of *rev* in Isabelle is not tail recursive, because it calls (@) after it calls *rev* in the second equation.

Write an alternative implementation of the reverse function that is tail recursive.

---

<sup>1</sup>See [https://en.wikipedia.org/wiki/Tail\\_call](https://en.wikipedia.org/wiki/Tail_call). for a quick review.

Hint: recall that you can most of the times derive a tail recursive function using an accumulator argument.

**fun** *rev\_tr*::'*a list*  $\Rightarrow$  '*a list* **where**

Show that the two implementations are equivalent.

**lemma** *rev\_tr xs = rev xs*